

DataFrame, pandas y NumPy

Idea clave: antes de entrenar un modelo, necesitamos datos organizados. pandas nos ayuda a trabajar con tablas; NumPy nos ayuda a trabajar con números, vectores y matrices.

1. Por qué necesitamos trabajar con datos en Machine Learning

El Machine Learning se basa en aprender patrones a partir de datos. Esos datos pueden venir de muchas fuentes: archivos CSV, bases de datos, formularios, sensores, hojas de cálculo o incluso imágenes, texto y audio transformados previamente en números.

Sin embargo, los datos casi nunca llegan perfectos. Pueden tener errores, valores vacíos, columnas mal nombradas, textos mezclados con números, información duplicada o formatos distintos dentro de una misma columna.

Por eso, antes de entrenar un modelo, normalmente hay una fase muy importante llamada preparación de datos. En esta fase intentamos convertir los datos originales en una estructura clara, limpia y útil para el análisis.

- Cargar los datos.
- Entender qué columnas y filas existen.
- Detectar valores vacíos o errores.
- Filtrar, ordenar y resumir información.
- Crear nuevas columnas.
- Separar variables de entrada y variable objetivo.

2. Qué es pandas

pandas es una biblioteca de Python especializada en el trabajo con datos estructurados. Un dato estructurado es aquel que se puede organizar en forma de tabla, con filas y columnas.

Este tipo de estructura es muy común en Machine Learning, porque muchos problemas se representan como tablas. Por ejemplo, una tabla puede contener alumnos, clientes, viviendas, productos, pacientes o transacciones.

Cada fila suele representar un ejemplo concreto. Cada columna suele representar una característica del ejemplo. En Machine Learning, a estas columnas también se les suele llamar variables, atributos o features.

- Fila: un registro o ejemplo.
- Columna: una variable, atributo o feature.
- Tabla: conjunto organizado de observaciones.

3. Qué es un DataFrame

Un DataFrame es la estructura principal de pandas. Podemos imaginarlo como una tabla inteligente de Python: se parece a una hoja de cálculo, pero se manipula mediante código.

Un DataFrame tiene filas, columnas, índices, tipos de datos y operaciones para filtrar, ordenar, agrupar y analizar información.

La ventaja es que podemos trabajar con conjuntos de datos grandes de forma mucho más cómoda que usando listas sueltas. Además, muchas operaciones que serían largas de programar manualmente ya están preparadas en pandas.

```
import pandas as pd

datos = {
    "nombre": ["Ana", "Luis", "Marta"],
    "edad": [16, 17, 16],
    "nota": [8.5, 4.2, 7.1],
    "aprobado": [True, False, True]
}

df = pd.DataFrame(datos)

print(df)
```

Resultado aproximado del DataFrame

Al ejecutar el código anterior, pandas muestra una tabla con un índice a la izquierda y las columnas definidas en el diccionario. El índice aparece automáticamente como 0, 1, 2, etc.

	nombre	edad	nota	aprobado
0	Ana	16	8.5	True
1	Luis	17	4.2	False
2	Marta	16	7.1	True

4. Filas, columnas e índice

En un DataFrame, cada fila representa un registro, cada columna representa una variable y el índice identifica cada fila.

pandas reconoce el tipo de dato de cada columna: textos, enteros, decimales, booleanos o valores vacíos. Esto es muy importante porque no todas las operaciones sirven para todos los tipos de datos.

- nombre: columna de texto.
- edad: columna de números enteros.
- nota: columna de números decimales.
- aprobado: columna booleana.

5. DataFrame y Machine Learning

En Machine Learning, un DataFrame suele representar un conjunto de datos. Si queremos predecir si un alumno aprobará, podemos tener columnas como horas de estudio, asistencia, nota anterior y una columna final que indique si aprueba.

Normalmente distinguimos entre variables de entrada y variable objetivo. A las variables de entrada se les suele llamar X. A la variable que queremos predecir se le suele llamar y.

Esta separación es fundamental: el modelo usa X para aprender patrones y compara sus predicciones con y durante el entrenamiento.

```
X = df[["horas_estudio", "asistencia", "nota_anterior"]]
y = df["aprueba"]
```

6. Operaciones básicas con DataFrames

Estas operaciones permiten hacer una primera exploración del conjunto de datos. Antes de entrenar un modelo conviene saber cuántos datos hay, qué columnas existen y qué tipos de valores contienen.

Esta fase inicial se llama a menudo análisis exploratorio de datos. No consiste todavía en entrenar un modelo, sino en comprender el material con el que vamos a trabajar.

- df.head(): muestra las primeras filas.
- df.tail(): muestra las últimas filas.
- df.info(): muestra información general.
- df.describe(): muestra estadísticas básicas.
- df.columns: muestra los nombres de las columnas.
- df.shape: muestra el número de filas y columnas.
- Si df.shape devuelve (100, 5), significa que hay 100 filas y 5 columnas.

7. Seleccionar columnas

Muchas veces no necesitamos todas las columnas del DataFrame. Podemos seleccionar una columna concreta o varias columnas para trabajar solo con la información necesaria.

```
df["nota"]

df[["nombre", "nota"]]
```

8. Filtrar filas

Filtrar significa quedarnos solo con las filas que cumplen una condición. Es una operación fundamental para analizar datos.

En pandas, cuando combinamos condiciones usamos & para and, | para or y ~ para not. Cada condición debe escribirse entre paréntesis.

Esto puede resultar extraño al principio porque en Python básico se usa and, or y not. En pandas se usan estos símbolos porque la condición se aplica a muchas filas a la vez, no a un único valor.

```
df[df["nota"] >= 5]

df[df["edad"] < 17]

df[(df["nota"] >= 5) & (df["edad"] < 18)]
```

9. Valores nulos o datos faltantes

En los datos reales es muy común encontrar valores vacíos. En pandas suelen aparecer como NaN. Un NaN indica que falta un dato o que no está disponible.

Esto importa mucho porque muchos modelos de Machine Learning no pueden entrenarse correctamente si hay datos faltantes. Antes de entrenar, debemos decidir si eliminamos esas filas, si rellenamos los huecos o si investigamos por qué faltan los datos.

- `df.isnull()`: detecta valores nulos.
- `df.isnull().sum()`: cuenta valores nulos por columna.
- `df.dropna()`: elimina filas con valores nulos.
- `df.fillna(0)`: rellena valores nulos con un valor concreto.

10. Crear nuevas columnas

Crear nuevas columnas permite transformar los datos originales en información más útil. Esta fase se relaciona con la ingeniería de características, una parte muy importante del trabajo previo al entrenamiento de modelos.

Por ejemplo, a partir de una columna `nota` podemos crear una columna `aprobado`. En lugar de guardar manualmente `True` o `False`, dejamos que pandas lo calcule usando una condición.

```
df["nota_final"] = df["nota"] + 1  
  
df["aprobado"] = df["nota"] >= 5
```

11. Qué es NumPy

NumPy es una biblioteca de Python especializada en cálculo numérico. Su estructura principal es el `array`, una estructura parecida a una lista, pero optimizada para trabajar con muchos números de forma rápida.

NumPy es importante porque muchas bibliotecas de Machine Learning trabajan internamente con `arrays`, `vectores` y `matrices`.

Una lista normal de Python puede guardar muchos tipos de datos diferentes. Un `array` de NumPy, en cambio, está pensado para cálculo numérico eficiente y suele contener datos del mismo tipo.

```
import numpy as np  
  
numeros = np.array([1, 2, 3, 4, 5])  
  
print(numeros)
```

12. Relación entre pandas y NumPy

pandas y NumPy suelen trabajar juntos. pandas nos ayuda a organizar y preparar los datos en forma de tabla. NumPy aporta la base matemática para trabajar con números, vectores y matrices.

Muchas veces, un `DataFrame` de pandas acaba convirtiéndose en `arrays` de NumPy para entrenar modelos.

Una forma sencilla de verlo es esta: pandas es cómodo para mirar y limpiar datos; NumPy es muy eficiente para hacer cálculos numéricos. En Machine Learning necesitamos ambas cosas.

```
valores = df[["edad", "nota"]].values
```

13. Preguntas antes de entrenar un modelo

Antes de aplicar Machine Learning, no basta con tener datos. Hay que mirarlos, entenderlos y prepararlos. pandas y NumPy son herramientas para responder preguntas básicas antes de entrenar.

Si no hacemos esta fase, podemos entrenar un modelo con errores, columnas irrelevantes, valores vacíos o datos mal interpretados. Un buen modelo empieza con una buena preparación de datos.

- Cuántos datos tengo.
- Qué columnas hay.
- Qué tipo de datos contiene cada columna.
- Hay valores vacíos o errores.
- Qué columna quiero predecir.
- Qué columnas usaré para predecir.
- Los datos están en formato numérico.
- Necesito transformar alguna columna.

14. Idea final

Un modelo de Machine Learning no empieza aprendiendo directamente. Primero necesita datos organizados.

pandas permite convertir un conjunto de datos en una tabla manejable. El DataFrame es esa tabla inteligente.

NumPy aporta la base matemática para trabajar con números, vectores y matrices.

Por eso, antes de entrenar modelos, tenemos que aprender a mirar, limpiar y transformar los datos.

Resumen final: un modelo de Machine Learning no empieza aprendiendo directamente. Primero necesita datos organizados, revisados y transformados. El DataFrame es la tabla inteligente con la que pandas facilita ese trabajo, y NumPy aporta la base numérica sobre la que se apoyan muchos modelos.